

Universal Function Approximation, Backpropagation via Computational Graphs, and Non-Convex Optimization

A Unified Mathematical, Information-Theoretic, and Physical Foundation of
Feedforward Neural Networks (MLP)

Contents

1	The Multi-Layer Perceptron (MLP) Paradigm	2
1.1	Universal Approximation Theorem	2
2	Computational Graphs and Vectorized Backpropagation	2
2.1	Batch-First Matrix Formulation	3
2.2	Common Loss Functions	3
2.3	Derivation of Backpropagation via Vectorized Chain Rule	3
3	Activation Functions and Gradient Dynamics	4
3.1	Activation Definitions and Derivatives	4
3.2	The Vanishing and Exploding Gradient Problem	5
4	Optimization Algorithms on Non-Convex Loss Surfaces	5
4.1	Stochastic Gradient Descent (SGD) with Momentum	6
4.2	Adaptive Moment Estimation (Adam)	6
5	Regularization and Internal Covariate Shift	6
5.1	Dropout (Inverted Dropout)	6
5.2	Batch Normalization	7
6	Information-Theoretic and Physical Analogies	7
6.1	Information-Theoretic View: The Information Bottleneck Principle	7
6.2	Physical Analogy: Langevin Dynamics and Energy Landscapes	7
7	Production-Grade Vectorized NumPy Implementation	8

1 The Multi-Layer Perceptron (MLP) Paradigm

Linear statistical estimators are constrained by their inability to model complex non-linear feature interactions without manual feature engineering. **Feedforward Neural Networks**, or **Multi-Layer Perceptrons (MLPs)**, solve this by composing parameterized linear transformations with non-linear activation functions into hierarchical computational networks.

An MLP represents an acyclic directed graph mapping an input space $\mathcal{X} \subseteq \mathbb{R}^{d_0}$ to a target space $\mathcal{Y} \subseteq \mathbb{R}^{d_L}$ through a sequence of L hidden layer representations.

Definition 1.1 (Feedforward Neural Network Architecture). Let $L \in \mathbb{N}^+$ denote the total depth (number of layers) of the network, and let d_l represent the dimensionality (width) of layer $l \in \{1, 2, \dots, L\}$, with d_0 being the input dimension. A Feedforward Neural Network maps input vector $\mathbf{x} \in \mathbb{R}^{d_0}$ to output prediction $\mathbf{a}^{(L)} \in \mathbb{R}^{d_L}$ via the recurrence relation:

$$\mathbf{a}^{(0)} = \mathbf{x} \tag{1}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \in \mathbb{R}^{d_l}, \quad \text{for } l = 1, 2, \dots, L \tag{2}$$

$$\mathbf{a}^{(l)} = \sigma_l(\mathbf{z}^{(l)}) \in \mathbb{R}^{d_l}, \quad \text{for } l = 1, 2, \dots, L \tag{3}$$

where:

- $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix connecting layer $l - 1$ to layer l .
- $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ is the bias vector for layer l .
- $\mathbf{z}^{(l)}$ is the pre-activation state vector at layer l .
- $\sigma_l : \mathbb{R} \rightarrow \mathbb{R}$ is an element-wise non-linear activation function applied at layer l .
- $\mathbf{a}^{(l)}$ is the post-activation feature representation at layer l .

The total set of trainable parameters $\boldsymbol{\theta}$ is the union of all layer parameters:

$$\boldsymbol{\theta} = \left\{ \mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)} \right\} \tag{4}$$

1.1 Universal Approximation Theorem

The foundational theoretical justification for feedforward networks rests on the **Universal Approximation Theorem**, established by Cybenko (1989) for sigmoidal activations and expanded by Hornik (1991) for arbitrary non-constant, bounded activations.

Theorem 1.1 (Universal Approximation Theorem (Hornik, 1991)). Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous, non-constant, bounded activation function. Let $C(\mathcal{K})$ denote the space of continuous functions on a compact subset $\mathcal{K} \subset \mathbb{R}^{d_0}$. For any target function $f \in C(\mathcal{K})$ and any arbitrary precision threshold $\epsilon > 0$, there exists a finite width $M \in \mathbb{N}^+$ and a single-hidden-layer network $g(\mathbf{x}) = \sum_{j=1}^M v_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j)$ such that:

$$\sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - g(\mathbf{x})| < \epsilon \tag{5}$$

Remark 1.1 (Width vs. Depth Trade-off). While a single hidden layer of infinite width ($M \rightarrow \infty$) can approximate any continuous function, deeper networks ($L \gg 2$) require exponentially fewer total parameters to approximate compositional, hierarchical structures.

2 Computational Graphs and Vectorized Backpropagation

Supervised training of neural networks minimizes an empirical loss function $\mathcal{L}(\boldsymbol{\theta})$ over an i.i.d. dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$. Evaluating parameter gradients efficiently requires computing partial derivatives of the scalar loss with respect to every weight and bias matrix.

2.1 Batch-First Matrix Formulation

Let B represent the mini-batch size. Stacking input samples row-wise yields the batch input matrix $\mathbf{A}^{(0)} = \mathbf{X} \in \mathbb{R}^{B \times d_0}$. Transposing weight parameter representations yields the batch forward pass equations for $l = 1, 2, \dots, L$:

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)} \mathbf{W}^{(l)T} + \mathbf{1}_B \mathbf{b}^{(l)T} \in \mathbb{R}^{B \times d_l} \quad (6)$$

$$\mathbf{A}^{(l)} = \sigma_l \left(\mathbf{Z}^{(l)} \right) \in \mathbb{R}^{B \times d_l} \quad (7)$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$, and $\mathbf{1}_B \in \mathbb{R}^B$ is a vector of ones.

2.2 Common Loss Functions

1. Mean Squared Error (MSE) – Regression

For target matrix $\mathbf{Y} \in \mathbb{R}^{B \times d_L}$ and network prediction $\hat{\mathbf{Y}} = \mathbf{A}^{(L)}$:

$$\mathcal{L}_{\text{MSE}}(\mathbf{Y}, \mathbf{A}^{(L)}) = \frac{1}{2B} \left\| \mathbf{Y} - \mathbf{A}^{(L)} \right\|_F^2 = \frac{1}{2B} \sum_{i=1}^B \sum_{j=1}^{d_L} \left(Y_{ij} - A_{ij}^{(L)} \right)^2 \quad (8)$$

2. Categorical Cross-Entropy (CCE) – Multi-Class Classification

Combined with a Softmax output activation $A_{ij}^{(L)} = \frac{\exp(Z_{ij}^{(L)})}{\sum_{k=1}^{d_L} \exp(Z_{ik}^{(L)})}$:

$$\mathcal{L}_{\text{CCE}}(\mathbf{Y}, \mathbf{A}^{(L)}) = -\frac{1}{B} \sum_{i=1}^B \sum_{j=1}^{d_L} Y_{ij} \ln \left(A_{ij}^{(L)} \right)$$

2.3 Derivation of Backpropagation via Vectorized Chain Rule

****Backpropagation**** applies the multivariate chain rule backwards through the computational graph, propagating error signals from the output layer L to the input layer 1.

Definition 2.1 (Pre-Activation Error Delta). Define the error tensor $\Delta^{(l)}$ at layer l as the partial derivative of the batch scalar loss $\mathcal{L}$ with respect to the pre-activation state matrix $\mathbf{Z}^{(l)}$:

$$\Delta^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l)}} \in \mathbb{R}^{B \times d_l} \quad (10)$$

Theorem 2.1 (Vectorized Backpropagation Equations). The error deltas, parameter gradients, and backward error propagation across layers satisfy the exact matrix relations:

$$\text{Output Layer Error Delta: } \Delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(L)}} \odot \sigma'_L \left(\mathbf{Z}^{(L)} \right) \quad (11)$$

$$\text{Hidden Layer Recurrence: } \Delta^{(l)} = \left(\Delta^{(l+1)} \mathbf{W}^{(l+1)} \right) \odot \sigma'_l \left(\mathbf{Z}^{(l)} \right) \quad (12)$$

$$\text{Weight Matrix Gradient: } \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \Delta^{(l)T} \mathbf{A}^{(l-1)} \in \mathbb{R}^{d_l \times d_{l-1}} \quad (13)$$

$$\text{Bias Vector Gradient: } \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \sum_{i=1}^B \Delta_{i,:}^{(l)T} \in \mathbb{R}^{d_l} \quad (14)$$

where $\odot$ denotes the Hadamard (element-wise) matrix product.

Proof. By the multivariate chain rule, consider element $Z_{ij}^{(l)}$ for sample i and neuron j :

$$\Delta_{ij}^{(l)} = \frac{\partial \mathcal{L}}{\partial Z_{ij}^{(l)}} = \sum_{k=1}^{d_{l+1}} \frac{\partial \mathcal{L}}{\partial Z_{ik}^{(l+1)}} \frac{\partial Z_{ik}^{(l+1)}}{\partial Z_{ij}^{(l)}} = \sum_{k=1}^{d_{l+1}} \Delta_{ik}^{(l+1)} \frac{\partial Z_{ik}^{(l+1)}}{\partial Z_{ij}^{(l)}} \quad (15)$$

Expanding $Z_{ik}^{(l+1)} = \sum_{m=1}^{d_l} A_{im}^{(l)} W_{km}^{(l+1)} + b_k^{(l+1)} = \sum_{m=1}^{d_l} \sigma_l(Z_{im}^{(l)}) W_{km}^{(l+1)} + b_k^{(l+1)}$:

$$\frac{\partial Z_{ik}^{(l+1)}}{\partial Z_{ij}^{(l)}} = W_{kj}^{(l+1)} \sigma'_l(Z_{im}^{(l)}) \quad (16)$$

Substituting this derivative back into the sum:

$$\Delta_{ij}^{(l)} = \sum_{k=1}^{d_{l+1}} \Delta_{ik}^{(l+1)} W_{kj}^{(l+1)} \sigma'_l(Z_{im}^{(l)}) = \left(\sum_{k=1}^{d_{l+1}} \Delta_{ik}^{(l+1)} W_{kj}^{(l+1)} \right) \sigma'_l(Z_{ij}^{(l)}) \quad (17)$$

In matrix notation, this matches Equation (12): $\Delta^{(l)} = (\Delta^{(l+1)} \mathbf{W}^{(l+1)}) \odot \sigma'_l(\mathbf{Z}^{(l)})$.

Similarly, applying the chain rule for weight element $W_{jk}^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \sum_{i=1}^B \frac{\partial \mathcal{L}}{\partial Z_{ij}^{(l)}} \frac{\partial Z_{ij}^{(l)}}{\partial W_{jk}^{(l)}} = \sum_{i=1}^B \Delta_{ij}^{(l)} A_{ik}^{(l-1)} = \left(\Delta^{(l)T} \mathbf{A}^{(l-1)} \right)_{jk} \quad (18)$$

This establishes Equation (13). Equation (14) follows directly by differentiating with respect to bias $b_j^{(l)}$. $\square$

Remark 2.1 (Softmax with Categorical Cross-Entropy Simplification). When pairing a Softmax output layer with Categorical Cross-Entropy loss, the combined gradient simplifies to the exact difference between predictions and ground-truth targets:

$$\Delta^{(L)} = \frac{1}{B} \left(\mathbf{A}^{(L)} - \mathbf{Y} \right) \quad (19)$$

This cancels potential division-by-zero singularities during backpropagation.

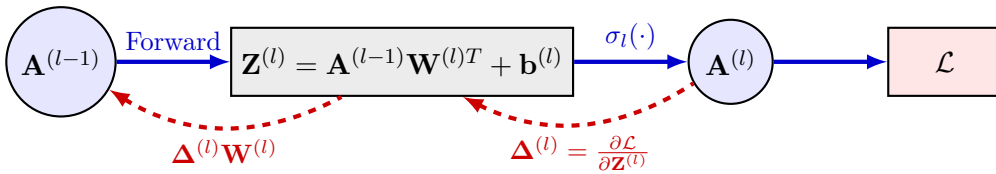


Figure 1: Forward activation flow vs. backward error propagation along a single layer of a computational graph.

3 Activation Functions and Gradient Dynamics

Activation functions introduce non-linearity, dictating the expressive capacity and gradient propagation stability of deep networks.

3.1 Activation Definitions and Derivatives

1. Logistic Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x) (1 - \sigma(x)) \in (0, 0.25] \quad (20)$$

2. Hyperbolic Tangent (Tanh)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh'(x) = 1 - \tanh^2(x) \in (0, 1] \quad (21)$$

3. Rectified Linear Unit (ReLU)

$$\text{ReLU}(x) = \max(0, x), \quad \text{ReLU}'(x) = \mathbb{I}(x > 0) \quad (22)$$

4. Gaussian Error Linear Unit (GELU)

$$\text{GELU}(x) = x\Phi(x) = xP(X \leq x), \quad X \sim \mathcal{N}(0, 1) \quad (23)$$

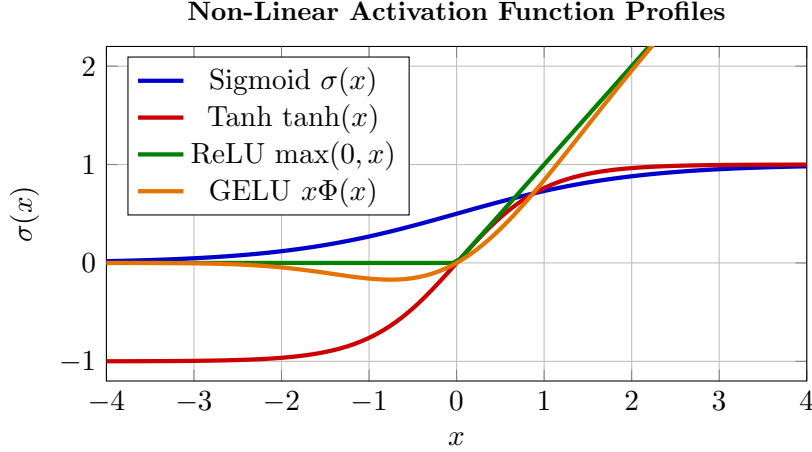


Figure 2: Comparison of primary neural network activation functions.

3.2 The Vanishing and Exploding Gradient Problem

Deep networks ($L \gg 1$) suffer from unstable gradient dynamics during backpropagation. Unrolling the recurrence relation for $\Delta^{(1)}$ back to layer L :

$$\Delta^{(1)} = \Delta^{(L)} \prod_{l=1}^{L-1} \left(\mathbf{W}^{(l+1)} \odot \sigma'_l(\mathbf{Z}^{(l)}) \right) \quad (24)$$

*****Vanishing Gradients:**** If activations saturate ($\sigma'(x) \rightarrow 0$), as in deep Sigmoid/Tanh networks where $\sigma'(x) \leq 0.25$, or if singular values of weight matrices satisfy $\sigma_{\max}(\mathbf{W}) < 1$, the product decays exponentially to zero: $\|\Delta^{(1)}\| \rightarrow 0$. Early layers stop updating, halting learning. *****Exploding Gradients:**** If $\sigma_{\max}(\mathbf{W}) > 1$ without saturating derivatives, $\|\Delta^{(1)}\| \rightarrow \infty$, causing numeric overflow ('NaN').

ReLU mitigates vanishing gradients along positive regimes ($\text{ReLU}'(x) = 1$), keeping gradient magnitudes constant across arbitrarily deep layers.

4 Optimization Algorithms on Non-Convex Loss Surfaces

Neural network loss landscapes are highly non-convex, characterized by vast saddle points, ill-conditioned curvature ravines, and local minima. Optimization relies on first-order stochastic gradient variants.

4.1 Stochastic Gradient Descent (SGD) with Momentum

Classical SGD suffers from high variance and oscillations in ill-conditioned ravines. ****Classical Momentum**** accumulates an exponentially decaying moving average of past gradients to accelerate along directions of consistent curvature.

Definition 4.1 (SGD with Nesterov Momentum). Let $\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$ be the batch gradient at step t , $\eta > 0$ be the learning rate, and $\gamma \in [0, 1)$ be the momentum coefficient:

$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t - \gamma \mathbf{v}_{t-1}) \quad (25)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t \quad (26)$$

4.2 Adaptive Moment Estimation (Adam)

****Adam**** (Kingma & Ba, 2014) combines adaptive learning rates for individual parameters (from RMSprop) with gradient momentum. It tracks exponentially decaying averages of past gradients ($\mathbf{m}_t$) and squared gradients ($\mathbf{v}_t$).

Definition 4.2 (Adam Optimization Update). Given hyperparameter defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and gradient $\mathbf{g}_t$: . Update biased 1st moment estimate: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$

Update biased 2nd raw moment estimate: $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$

Compute bias-corrected 1st moment estimate:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad (27)$$

Compute bias-corrected 2nd moment estimate:

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (28)$$

Update parameter vector:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \hat{\mathbf{m}}_t \quad (29)$$

5 Regularization and Internal Covariate Shift

High-capacity MLPs can memorize training noise. Regularization constrains capacity and stabilizes deep training.

5.1 Dropout (Inverted Dropout)

****Dropout**** (Srivastava et al., 2014) prevents co-adaptation of features by zeroing activations randomly during training with probability $p \in (0, 1)$.

During training, for layer l : . Sample a random Bernoulli mask vector $\mathbf{r}^{(l)} \sim \text{Bernoulli}(1 - p)$.

Mask and scale activation vector:

$$\tilde{\mathbf{a}}^{(l)} = \frac{1}{1 - p} \left(\mathbf{a}^{(l)} \odot \mathbf{r}^{(l)} \right) \quad (30)$$

Scaling by $\frac{1}{1-p}$ (Inverted Dropout) ensures the expected activation magnitude remains invariant, allowing test-time inference to run without modification ($\tilde{\mathbf{a}}_{\text{test}}^{(l)} = \mathbf{a}^{(l)}$).

5.2 Batch Normalization

Batch Normalization (Ioffe & Szegedy, 2015) reduces **Internal Covariate Shift**—the continuous shift in the distribution of layer inputs during training caused by parameter updates in preceding layers.

For a mini-batch activation tensor $\mathbf{Z}^{(l)} \in \mathbb{R}^{B \times d_l}$ along feature dimension j : . Compute mini-batch mean: $\mu_j = \frac{1}{B} \sum_{i=1}^B Z_{ij}^{(l)}$

Compute mini-batch variance: $\sigma_j^2 = \frac{1}{B} \sum_{i=1}^B (Z_{ij}^{(l)} - \mu_j)^2$

Normalize pre-activations:

$$\hat{Z}_{ij}^{(l)} = \frac{Z_{ij}^{(l)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad (31)$$

Apply learnable affine scale (γ_j) and shift (β_j) parameters:

$$\tilde{Z}_{ij}^{(l)} = \gamma_j \hat{Z}_{ij}^{(l)} + \beta_j \quad (32)$$

6 Information-Theoretic and Physical Analogies

6.1 Information-Theoretic View: The Information Bottleneck Principle

Tishby et al. (2015) formulated deep neural network training through the **Information Bottleneck (IB) Principle**.

Let X be the input variable, Y be the target, and $T_l = \mathbf{A}^{(l)}$ be the hidden representation at layer l . The network forms a Markov chain $Y \rightarrow X \rightarrow T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_L \rightarrow \hat{Y}$.

Each hidden layer attempts to trade off two competing mutual information quantities:

$$\min_{p(t|x)} \mathcal{L}_{\text{IB}} = I(X; T_l) - \beta I(T_l; Y) \quad (33)$$

Compression ($I(X; T_l) \rightarrow 0$): Minimizing $I(X; T_l)$ forces early hidden layers to compress and discard irrelevant nuisance variations in input space X . **Sufficiency** ($I(T_l; Y) \rightarrow I(X; Y)$): Maximizing $I(T_l; Y)$ ensures the representation retains information necessary to predict target Y .

Deep networks naturally split training into two temporal phases: an initial **fitting phase** (where $I(T; Y)$ rises rapidly) followed by a longer **compression phase** (where $I(X; T)$ drops as parameters diffuse).

6.2 Physical Analogy: Langevin Dynamics and Energy Landscapes

Mini-batch SGD optimization on non-convex loss surfaces maps onto the physical trajectory of a particle undergoing **Brownian Motion** inside a complex potential energy landscape $V(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta})$.

Sampling mini-batches creates stochastic gradient noise $\boldsymbol{\epsilon}_t = \mathbf{g}_t - \nabla \mathcal{L}(\boldsymbol{\theta})$. By the Central Limit Theorem, $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{C}(\boldsymbol{\theta}))$, where covariance $\mathbf{C}$ scales with learning rate η and inverse batch size $1/B$.

The parameter trajectory follows a continuous **Overdamped Langevin Stochastic Differential Equation**:

$$d\boldsymbol{\theta}_t = -\nabla \mathcal{L}(\boldsymbol{\theta}_t) dt + \sqrt{2T_{\text{eff}}} d\mathbf{W}_t \quad (34)$$

where $d\mathbf{W}_t$ is a Wiener process, and $T_{\text{eff}} = \frac{\eta}{2B}$ acts as an **Effective Thermal Temperature**: **High Thermal Fluctuation** ($T_{\text{eff}} \uparrow$, large η , small B): The system escapes sharp, fragile local minima (high kinetic energy crossing energy barriers). **Low Thermal Annealing** ($T_{\text{eff}} \downarrow$, small η , large B): The system settles into wide, flat minima that generalize well.

7 Production-Grade Vectorized NumPy Implementation

The following Python script provides a production-grade implementation of a Feedforward Neural Network built strictly from scratch in NumPy. It includes vectorized forward/backward passes, Dense layers, ReLU, Softmax, Categorical Cross-Entropy, and the Adam Optimizer.

```
1 import numpy as np
2
3 class Layer_Dense:
4     """Fully connected dense layer with Xavier/Glorot normal initialization
5     ."""
6     def __init__(self, input_dim: int, output_dim: int):
7         # Xavier/Glorot initialization scale
8         init_scale = np.sqrt(2.0 / (input_dim + output_dim))
9         self.weights = np.random.randn(input_dim, output_dim) * init_scale
10        self.biases = np.zeros((1, output_dim))
11
12        # Gradients
13        self.dweights = None
14        self.dbiases = None
15
16        # Inputs cache for backward pass
17        self.inputs = None
18
19    def forward(self, inputs: np.ndarray) -> np.ndarray:
20        """Forward pass:  $Z = A_{prev} @ W + b$ """
21        self.inputs = inputs
22        return np.dot(inputs, self.weights) + self.biases
23
24    def backward(self, dvalues: np.ndarray) -> np.ndarray:
25        """
26        Backward pass: computes gradients dW, db, and propagates dA_prev.
27        dvalues is  $\Delta^{(l)}$  of shape (B, d_l).
28        """
29        batch_size = dvalues.shape[0]
30        # dW = (1/B) * A_prev^T @ Delta
31        self.dweights = np.dot(self.inputs.T, dvalues) / batch_size
32        # db = (1/B) * sum(Delta, axis=0)
33        self.dbiases = np.sum(dvalues, axis=0, keepdims=True) / batch_size
34        # dA_prev = Delta @ W^T
35        return np.dot(dvalues, self.weights.T)
36
37 class Activation_ReLU:
38     """Rectified Linear Unit (ReLU) activation function."""
39     def __init__(self):
40         self.inputs = None
41
42     def forward(self, inputs: np.ndarray) -> np.ndarray:
43         self.inputs = inputs
44         return np.maximum(0, inputs)
45
46     def backward(self, dvalues: np.ndarray) -> np.ndarray:
47         """Backward pass: propagates dvalues * I(inputs > 0)."""
48         dinputs = dvalues.copy()
49         dinputs[self.inputs <= 0] = 0.0
50         return dinputs
51
52
53 class Activation_Softmax_Loss_CategoricalCrossentropy:
```

```

54     """
55     Combined Softmax Activation and Categorical Cross-Entropy Loss for
56     numerical stability during backward pass evaluation.
57     """
58     def __init__(self):
59         self.output = None
60
61     def forward(self, inputs: np.ndarray, y_true: np.ndarray) -> float:
62         """
63         Forward pass evaluating stable Softmax and Mean Cross-Entropy.
64         inputs shape: (B, C), y_true shape: (B, C) one-hot or (B,) class
65         indices.
66         """
67         # Stable Softmax via Log-Sum-Exp constant subtraction
68         exp_values = np.exp(inputs - np.max(inputs, axis=1, keepdims=True))
69         probabilities = exp_values / np.sum(exp_values, axis=1, keepdims=
70 True)
71         self.output = probabilities
72
73         # Clip probabilities to prevent log(0)
74         probs_clipped = np.clip(probabilities, 1e-15, 1.0 - 1e-15)
75
76         # Categorical cross-entropy
77         if len(y_true.shape) == 1:
78             correct_log_probs = probs_clipped[range(len(inputs)), y_true]
79         else:
80             correct_log_probs = np.sum(probs_clipped * y_true, axis=1)
81
82         return float(np.mean(-np.log(correct_log_probs)))
83
84     def backward(self, dvalues: np.ndarray, y_true: np.ndarray) -> np.
85 ndarray:
86         """
87         Combined Softmax + Cross-Entropy gradient:
88         Delta^(L) = (A^(L) - Y)
89         """
90         samples = len(dvalues)
91         if len(y_true.shape) == 2:
92             y_true = np.argmax(y_true, axis=1)
93
94         dinputs = dvalues.copy()
95         # Direct analytical gradient subtraction
96         dinputs[range(samples), y_true] -= 1.0
97         return dinputs
98
99 class Optimizer_Adam:
100     """Production-grade Adam (Adaptive Moment Estimation) Optimizer."""
101     def __init__(self, learning_rate: float = 0.001, beta_1: float = 0.9,
102         beta_2: float = 0.999, epsilon: float = 1e-8):
103         self.lr = learning_rate
104         self.beta_1 = beta_1
105         self.beta_2 = beta_2
106         self.epsilon = epsilon
107         self.iterations = 0
108
109         # Moment tracking dictionaries
110         self.m_weights = {}
111         self.v_weights = {}
112         self.m_biases = {}

```

```

111         self.v_biases = {}
112
113     def update_params(self, layer: Layer_Dense, layer_idx: int):
114         """Updates weights and biases for a single layer using Adam
115         dynamics."""
116         if layer_idx not in self.m_weights:
117             self.m_weights[layer_idx] = np.zeros_like(layer.weights)
118             self.v_weights[layer_idx] = np.zeros_like(layer.weights)
119             self.m_biases[layer_idx] = np.zeros_like(layer.biases)
120             self.v_biases[layer_idx] = np.zeros_like(layer.biases)
121
122         t = self.iterations + 1
123
124         # --- Update Weight Moments ---
125         self.m_weights[layer_idx] = self.beta_1 * self.m_weights[layer_idx]
126         + (1.0 - self.beta_1) * layer.dweights
127         self.v_weights[layer_idx] = self.beta_2 * self.v_weights[layer_idx]
128         + (1.0 - self.beta_2) * (layer.dweights ** 2)
129
130         m_w_corrected = self.m_weights[layer_idx] / (1.0 - self.beta_1 ** t)
131         v_w_corrected = self.v_weights[layer_idx] / (1.0 - self.beta_2 ** t)
132
133         layer.weights -= self.lr * m_w_corrected / (np.sqrt(v_w_corrected)
134         + self.epsilon)
135
136         # --- Update Bias Moments ---
137         self.m_biases[layer_idx] = self.beta_1 * self.m_biases[layer_idx] +
138         (1.0 - self.beta_1) * layer.dbiases
139         self.v_biases[layer_idx] = self.beta_2 * self.v_biases[layer_idx] +
140         (1.0 - self.beta_2) * (layer.dbiases ** 2)
141
142         m_b_corrected = self.m_biases[layer_idx] / (1.0 - self.beta_1 ** t)
143         v_b_corrected = self.v_biases[layer_idx] / (1.0 - self.beta_2 ** t)
144
145         layer.biases -= self.lr * m_b_corrected / (np.sqrt(v_b_corrected) +
146         self.epsilon)
147
148     def post_update(self):
149         self.iterations += 1
150
151 class MultiLayerPerceptron:
152     """Full Sequential MLP Container supporting training and inference."""
153     def __init__(self):
154         self.layers = []
155         self.softmax_loss = Activation_Softmax_Loss_CategoricalCrossentropy()
156
157     def add_dense(self, input_dim: int, output_dim: int):
158         self.layers.append(Layer_Dense(input_dim, output_dim))
159         self.layers.append(Activation_ReLU())
160
161     def forward(self, X: np.ndarray) -> np.ndarray:
162         """Executes forward pass across all internal dense and activation
163         layers."""
164         out = X
165         for layer in self.layers:
166             out = layer.forward(out)

```

```

160         return out
161
162     def fit(self, X: np.ndarray, y: np.ndarray, epochs: int = 200,
163            batch_size: int = 64, learning_rate: float = 0.01):
164         """Trains network parameters using Mini-Batch Adam SGD."""
165         optimizer = Optimizer_Adam(learning_rate=learning_rate)
166         num_samples = X.shape[0]
167
168         for epoch in range(1, epochs + 1):
169             # Shuffle mini-batches
170             indices = np.arange(num_samples)
171             np.random.shuffle(indices)
172             X_shuffled = X[indices]
173             y_shuffled = y[indices]
174
175             epoch_loss = 0.0
176             num_batches = int(np.ceil(num_samples / batch_size))
177
178             for b in range(num_batches):
179                 start = b * batch_size
180                 end = min(start + batch_size, num_samples)
181
182                 X_batch = X_shuffled[start:end]
183                 y_batch = y_shuffled[start:end]
184
185                 # 1. Forward Pass
186                 logits = self.forward(X_batch)
187                 loss = self.softmax_loss.forward(logits, y_batch)
188                 epoch_loss += loss * (end - start)
189
190                 # 2. Backward Pass
191                 dinputs = self.softmax_loss.backward(self.softmax_loss.
output, y_batch)
192                 for layer in reversed(self.layers):
193                     dinputs = layer.backward(dinputs)
194
195                 # 3. Parameter Update (Adam)
196                 for idx, layer in enumerate(self.layers):
197                     if isinstance(layer, Layer_Dense):
198                         optimizer.update_params(layer, idx)
199
200                 optimizer.post_update()
201
202                 if epoch == 1 or epoch % 20 == 0:
203                     avg_loss = epoch_loss / num_samples
204                     preds = np.argmax(self.softmax_loss.output, axis=1)
205                     y_true_labels = y_batch if len(y_batch.shape) == 1 else np.
argmax(y_batch, axis=1)
206                     acc = np.mean(preds == y_true_labels) * 100
207                     print(f"Epoch {epoch:03d}/{epochs} - Batch Loss: {avg_loss
:.4f} - Last Batch Acc: {acc:.2f}%")
208
209
210 if __name__ == "__main__":
211     np.random.seed(42)
212
213     # Generate Synthetic Non-Linear Multi-Class Spiral Dataset
214     N_per_class = 200
215     K_classes = 3
216     d_in = 2

```

```

217 X_data = np.zeros((N_per_class * K_classes, d_in))
218 y_data = np.zeros(N_per_class * K_classes, dtype=int)
219
220
221 for class_number in range(K_classes):
222     ix = range(N_per_class * class_number, N_per_class * (class_number
+ 1))
223     r = np.linspace(0.0, 1.0, N_per_class) # Radius
224     t = np.linspace(class_number * 4.0, (class_number + 1) * 4.0,
N_per_class) + np.random.randn(N_per_class) * 0.2
225     X_data[ix] = np.c_[r * np.sin(t), r * np.cos(t)]
226     y_data[ix] = class_number
227
228 # Instantiate 3-Layer MLP Architecture (2 -> 64 -> 32 -> 3)
229 mlp = MultiLayerPerceptron()
230 mlp.add_dense(input_dim=2, output_dim=64)
231 mlp.add_dense(input_dim=64, output_dim=32)
232 mlp.layers.append(Layer_Dense(input_dim=32, output_dim=3)) # Final
logit layer
233
234 print("=== MULTI-LAYER PERCEPTRON (MLP) INITIALIZATION SUCCESSFUL ===")
235 print(f"Total Dataset Size: {X_data.shape[0]} samples | Feature Dim: {
d_in} | Classes: {K_classes}")
236 print("Beginning Training Loop with Adam Optimizer...\n")
237
238 mlp.fit(X_data, y_data, epochs=100, batch_size=64, learning_rate=0.01)

```

Listing 1: Production-grade Vectorized NumPy implementation of a Multi-Layer Perceptron.